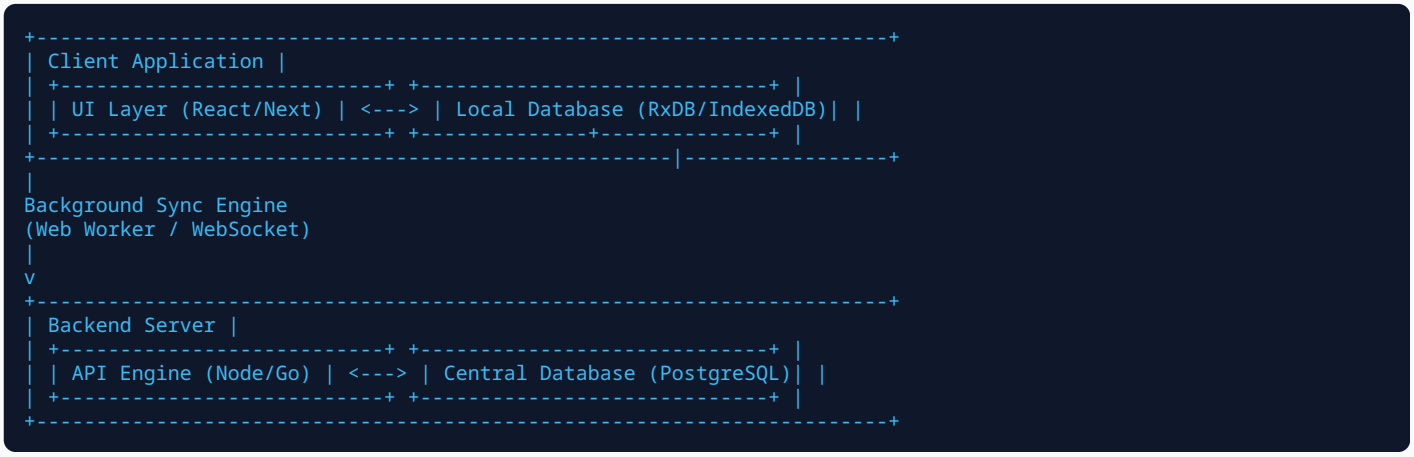
# ICT Equipment & Borrowing Management System

Technical Project Guide & Execution Blueprint for Claude Code

**CLAUDE CODE READY SPECIFICATION**

## 1. System Architecture Strategy

The system is engineered as a **Local-First Hybrid Application** capable of full operational functionality in offline environments. Changes made offline are recorded locally in IndexedDB (via RxDB) and auto-synchronized with PostgreSQL upon restoring internet connectivity.




## 2. Comprehensive User Stories

| Epic | ID | User Role & Goal | Acceptance Criteria |
|---|---|---|---|
| Category Management | US-1.1<br>US-1.2<br>US-1.3 | Admin: Taxonomy & Attributes | • Create/Edit/Soft-delete categories.<br>• Custom dynamic metadata JSON fields per category.<br>• Default loan duration auto-applied to checkout logic. |
| Equipment Inventory | US-2.1<br>US-2.2<br>US-2.3<br>US-2.4 | Inventory Manager: Equipment Control | • Register equipment with Asset Tag, Serial No, Status, Location.<br>• Print and scan QR/Barcodes for fast audit.<br>• Track status transitions (Available, Borrowed, Maintenance, Decommissioned).<br>• Fast offline search and filtering. |
| Borrowing & Return | US-3.1<br>US-3.2<br>US-3.3<br>US-3.4 | Staff & Manager: Circulation | • Request hardware loans with target return dates.<br>• Approve/Issue items via barcode scanning.<br>• Process returns with condition assessment.<br>• Automated notification logic for overdue items. |
| Hybrid Offline Sync | US-4.1<br>US-4.2<br>US-4.3<br>US-4.4 | All Users: Sync Engine | • Full CRUD offline capability.<br>• Auto background sync on network reconnection.<br>• Server-Validation for item status conflicts.<br>• Visual indicator showing online/offline status & queue count. |

## 3. Database Schema Blueprint

```
-- PostgreSQL Core Schema

CREATE TABLE categories (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
```

```
    description TEXT,
    custom_fields JSONB DEFAULT '{}',
    default_loan_days INT DEFAULT 7,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP WITH TIME ZONE
);

CREATE TABLE equipment (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    asset_tag VARCHAR(100) UNIQUE NOT NULL,
    serial_number VARCHAR(100) UNIQUE,
    category_id UUID REFERENCES categories(id),
    model_name VARCHAR(255) NOT NULL,
    brand VARCHAR(100),
    status VARCHAR(50) DEFAULT 'AVAILABLE', -- AVAILABLE, BORROWED, MAINTENANCE, DECOMMISSIONED
    condition VARCHAR(50) DEFAULT 'GOOD',    -- NEW, GOOD, FAIR, DAMAGED
    location VARCHAR(255),
    specifications JSONB DEFAULT '{}',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP WITH TIME ZONE
);

CREATE TABLE borrowing_transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    equipment_id UUID REFERENCES equipment(id),
    borrower_name VARCHAR(255) NOT NULL,
    borrower_email VARCHAR(255) NOT NULL,
    issue_date TIMESTAMP WITH TIME ZONE NOT NULL,
    expected_return_date TIMESTAMP WITH TIME ZONE NOT NULL,
    actual_return_date TIMESTAMP WITH TIME ZONE,
    status VARCHAR(50) DEFAULT 'ISSUED', -- REQUESTED, ISSUED, RETURNED, OVERDUE, LOST
    notes TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP WITH TIME ZONE
);
```

## 4. Offline Sync & Conflict Handling Policy

**Sync Protocol Sequence:**

1. **Outbox Pattern:** Offline mutations write to RxDB and log mutation events into local `sync_queue`.
2. **Push Phase:** When online, batch POSTs pending queue items to `/api/sync/push`.
3. **Pull Phase:** Fetches server deltas via `/api/sync/pull?last_synced_at=TIMESTAMP`.
4. **Conflict Strategy:** General fields use Last-Write-Wins (LWW). Physical status transitions (e.g. double booking) enforce Server Validation and flag conflicts as `CONFLICT_PENDING_REVIEW`.

## 5. Executable Claude Code Prompts

Run the following prompts sequentially inside Claude Code terminal:

### Phase 1: Setup Framework & Schema

```
claude "Initialize Next.js App Router project with TypeScript, Tailwind CSS, and lucide-react. Set up Prisma / PostgreSQL ORM schemas for Categories, Equipment, BorrowingTransactions, and SyncLogs."
```

### Phase 2: Offline Store Engine

```
claude "Implement RxDB client database setup with IndexedDB adapter. Create schemas matching Equipment, Categories, and Borrowing. Implement a local Sync Queue module using Outbox pattern."
```

### Phase 3: Inventory & Category UI

```
claude "Build CRUD pages for Categories with dynamic JSON custom fields builder. Create Equipment management page with table search, offline filtering, and QR code scanner component."
```

### Phase 4: Circulation & Checkout Logic

```
claude "Build Borrowing/Return checkout workflow UI. Include quick scan checkout, duration calculations based on category defaults, return inspection form, and status badge indicators."
```

### Phase 5: Sync Engine API Routes

```
claude "Build /api/sync/push and /api/sync/pull endpoints. Implement conflict detection for overlapping borrowings and Last-Write-Wins for equipment metadata. Connect client sync engine to auto-trigger on network connection restoration."
```